

Setting Up a GitHub Repository, Start to Finish

For the `ligo_uto_analysis` (v2) project — picking up right where you left off, including a `.gitignore` built from your own files.

Read this top to bottom once. After that, jump straight to the "Quick Command List" at the end whenever you set up a future project — it's the same steps with no explanation attached, for when you just need a reminder.

Where you already are: you've run `git init` inside your project folder, and renamed the branch. Nothing has been committed yet. That's actually the ideal moment to add a `.gitignore` — once files are committed, "ignoring" them later takes extra cleanup steps.

Part 1 — What a `.gitignore` File Actually Does

Think of `git add` as "everything I put in this box gets shipped to GitHub." A `.gitignore` file is a note taped to the box that says "don't even look at these items on the shelf — they're not going in." Git quietly skips them. They still exist on your computer; they're just invisible to Git.

Why now, specifically: this file has to exist *before* your first `git add .`, or Git will already have some of these files in its history and "ignoring" them afterward won't remove them — it only stops tracking future changes to them. Since nothing is committed yet, you're at the perfect point to set this up cleanly.

Part 2 — Sorting Your Current Files

Based on what `git status` showed, here's the include/exclude call for each item, and the reasoning behind it.

Include — these get shipped to GitHub

- **`uto_ligo_analysis_v2.py`** — this is the actual pipeline script; the whole point of the repo.
- **`network_catalogs_v2.py`** — the network-download functions (File B). Source code you wrote, same reasoning.
- **`design_context/`** — your planning and spec documents. This is exactly what an assessor wants to see, because it shows your thinking, not just your output.
- **`working_guides/`** — your reference guides and workflow notes. Worth keeping in the repo as documentation — but skim them once for file paths or personal details before committing, since your own rule is to keep anything public PII-free.

Exclude — these stay off the shelf

- **`__pycache__/`** — Python's auto-generated bytecode cache. It rebuilds every time you run the script, so tracking it is like version-controlling a shadow — pointless, and it clutters every diff.

- **raw_csvs/** — downloaded LIGO data, not your own work. It's also almost certainly too big for GitHub (files over 100 MB are rejected outright), and anyone can re-fetch the same files from the source link in your README.
- **planet_bsps/** — the planetary ephemeris binary files for the future planet-lookup pass. Same reasoning as raw_csvs: large, downloaded, and regenerable — not something you authored.

Part 3 — More Examples, for Files You'll Add Later

These aren't in your folder yet, but will show up as the v2 script runs and as the project grows. Good to have the pattern ready in advance.

More things to exclude

- **checkpoints/** — the step-by-step .pkl checkpoint files. Large, and meant to be recreated by rerunning the script, not shipped with it.
- **outputs/** — result CSVs and the diagnostic_printouts folder. Same logic: this is what the script produces, not what you wrote.
- **catalogs/** — the dedicated cache folder for downloaded star/galaxy catalogs planned for v2. Same as raw_csvs: a downloaded cache, not your work.
- **ligo-env/ or venv/** — your virtual environment folder. A self-contained copy of installed packages, easily rebuilt with a requirements file, and often hundreds of MB.
- **.env** — if you ever keep an API key or token in a file like this, it must never be committed. GitHub scans for some known formats, but don't rely on that as your only safety net.
- **.DS_Store / Thumbs.db** — junk files your operating system creates inside folders. Harmless to you, meaningless to anyone else.
- **.vscode/ / .idea/** — personal code-editor settings, relevant only to your own machine.

More things to include

- Any **.py** file you wrote — that's the actual deliverable.
- **README.md** and any other documentation markdown describing the project.
- **requirements.txt**, once you make one — a short list of the packages needed to run the script (numpy, pandas, astropy, astroquery, pycbc). This is what lets someone rebuild your virtual environment without you shipping it.
- **LICENSE**, if you add one later.

Part 4 — Building the .gitignore File

Step 4.1 — Confirm you're in the project folder

```
pwd
```

This should print a path ending in `ligo_uto_analysis`. If it doesn't, `cd` into that folder first.

Step 4.2 — Open a new file called .gitignore

```
nano .gitignore
```

The leading dot makes this a "hidden" file on Linux — completely normal, and Git specifically looks for a file with exactly this name.

Step 4.3 — Type or paste in this content

```
# ==== Auto-generated / cache (Python rebuilds these automatically) ====
__pycache__/
*.pyc
.pytest_cache/
.mypy_cache/

# ==== Downloaded / regenerable data (not your work, often too big for GitHub) ====
raw_csvs/
planet_bsps/
catalogs/

# ==== Script-generated results (rebuilt by rerunning the pipeline) ====
checkpoints/
outputs/

# ==== Virtual environment (rebuilt with requirements.txt) ====
ligo-env/
venv/
.venv/

# ==== Secrets (never commit these) ====
.env

# ==== OS / editor clutter ====
.DS_Store
Thumbs.db
.vscode/
.idea/
```

A matching plain-text copy of this block is provided alongside this PDF, so you can copy-paste it directly instead of retyping it.

Step 4.4 — Save and exit

- `Ctrl+O` then `Enter` to save
- `Ctrl+X` to exit nano

Part 5 — Confirm It Worked

```
git status
```

The untracked-files list should now be shorter. You should see your `.py` files, `design_context/`, `working_guides/`, and `.gitignore` itself listed — but `__pycache__/`, `raw_csvs/`, and `planet_bsps/` should have disappeared from the list entirely. That disappearance is Git confirming the ignore rules are working, not an error.

Part 6 — Stage and Make Your First Commit

Step 6.1 — Stage everything that's left

```
git add .
```

Step 6.2 — Check the result

```
git status
```

Everything should now show as staged (green, or under "Changes to be committed").

Step 6.3 — Commit

```
git commit -m "Initial commit: v2 script skeleton"
```

A commit message is a short, present-tense note describing what this snapshot contains. You'll write a new one every time you commit — there's no fixed length or format, just something that will make sense to you later.

Part 7 — Create the Empty Repository on GitHub.com

- Log into github.com in your browser.
- Click the + icon in the top-right corner, then **New repository**.
- Give it a name — for example, `ligo_uto_analysis`.
- Choose **Public** or **Private** — your call, since this is heading to GitHub for assessment anyway, Public is the usual choice.
- Leave every checkbox on that page unticked — do **not** check "Add a README," "Add .gitignore," or "Choose a license." You already have files locally; if GitHub creates its own on the website, the two histories won't match and your first push will be rejected.
- Click **Create repository**.

GitHub will now show a page with some commands on it — those work too, but they do the same thing as Parts 8 and 9 below, so pick one path and follow it.

Part 8 — Connect Your Local Folder to That GitHub Repo

GitHub's page after creation shows a URL ending in `.git` — copy it, then run:

```
git remote add origin <paste the URL here>
```

Confirm it's linked:

```
git remote -v
```

You should see "origin" listed twice (fetch and push), both pointing at your new GitHub repository.

Part 9 — Push Your Code Up

```
git push -u origin uto_ligo_analysis_v2
```

The `-u` flag links this local branch to that remote branch as the default pairing — after this, a plain `git push` is enough.

If you're asked for a username and password: the username is your GitHub username. For the password field, paste your Personal Access Token (PAT) — GitHub no longer accepts your actual account password for this. Since `credential.helper store` is already set up from earlier, it should remember this after the first successful push and not ask again for a while.

Part 10 — Verify

Refresh the repository page in your browser. Your files should now be listed there — matching what's in your project folder, minus everything the `.gitignore` excluded.

Part 11 — Quick Command List (Cheat Sheet)

Same steps as above, no explanation — for next time, or for a future project.

```
cd <your project folder>
nano .gitignore           # paste the template, save, exit
git status                # confirm ignored items dropped off the list
git add .
git status                # confirm everything is staged
git commit -m "Initial commit"
git remote add origin <your GitHub repo URL>
git remote -v             # confirm it's linked
git push -u origin <branch name>
```

Troubleshooting

- **"remote origin already exists"** — you already ran `git remote add` once. Use `git remote set-url origin <url>` instead.
- **"failed to push some refs" / push rejected** — usually means the GitHub repo wasn't actually empty (a README got created on the website after all). The two histories don't match. Safest fix as a first-timer: delete that file from the GitHub website, then push again — or stop and ask before forcing anything, since force-pushing can overwrite history.
- **Authentication failed** — your Personal Access Token has likely expired or was mistyped. Generate a new one under GitHub Settings, then Developer settings, then Personal access tokens.
- **A file/folder you meant to ignore still shows up in git status** — it was probably already tracked before the `.gitignore` existed. Check with `git status`; if it's already staged from an earlier `git add .`, run `git rm -r --cached <name>` to un-stage it without deleting it from your disk, then check status again.